

Lab 3: Architectural Decision Justification

Dhareen G | SRN: PES1UG24AM084 | PES University, CSE (AI & ML)

Architecture Selection: We chose Layered Architecture for the Digital Campus Library Reservation Gateway.

Architectural choice. A three-layer style: Presentation (Library Portal UI), Business (Catalog Search, Authentication, Reservation Manager and Notification components) and Data (Library Database). Each layer only calls the layer directly below it, through the interfaces shown in the component diagram (ICatalogSearch, IAuthentication, IReservation, INotification, ICatalogData, IMemberData, IHoldData).

Two reasons

1. The workflow is naturally tiered. UC-01 (Search ISBN Catalog) already follows a presentation, rules, data pattern: the user enters a term, the system validates it, queries the catalog, then returns paginated results with availability status. Placing a hold adds business rules (login required, only CheckedOut titles, queue order). Each step lands in exactly one layer, so the design maps directly onto the use case and each component stays small and replaceable behind its interface.

2. It fits the size and consistency needs of a campus system. The gateway serves one institution with a small team, and availability and hold data must stay strictly consistent so two members cannot hold the same copy. Layered architecture keeps this as one deployable unit with one database. Microservices would add network calls, separate data stores and distributed transactions for a load that does not need independent scaling. A plain client-server design would mix validation and hold rules into the client or the database and make them harder to change.

Security advantage. The UI never talks to the database. Every request passes through the business layer, where the Authentication component enforces the rule that searching is open but placing a hold needs a logged-in member, and where input is validated (UC-01 step 2) before any query runs. Only the data layer holds database credentials, using a least-privilege account, so a compromised or malformed request cannot reach tables directly and SQL injection exposure is limited to one controlled access point.

Performance benefit. Catalog search is read-heavy. Layering lets us add caching of frequent search results at the business layer and an ISBN/title index at the data layer without touching the UI. Pagination keeps response sizes small, and calls between layers are in-process, so there is none of the network latency a microservices split would add to every search.

Trade-off accepted. Requests cross every layer, which adds a small overhead, and the layers cannot be scaled independently. For a single campus library this cost is minor, and the Notification component can be moved to its own service later because it is already isolated behind INotification.